장성웅

j03j03j03@naver.com
포트폴리오 sungwoong37.github.io/JSWphotfollio
GitHub github.com/sungwoong37

게임플레이 클라이언트 프로그래머

핵심 역량

- **출시 경험:** 3인 팀의 팀장이자 프로그래머로 2D 플랫포머 「Rat-errible Climb」을 스토브 인디에 출시, 3인 팀의 개발 담당으로 2~4인 협동 액션 「Digestor」를 스토브에 출시
- **게임플레이 시스템:** 상태머신 기반 함정 시스템, 서버 권위 구조의 온라인 협동 멀티플레이 구현
- **C++ · 언리얼 엔진 5:** 독학으로 장작불 시뮬레이션 「One More Log」 개발 중, 엔진과 분리한 C++17 연소 코어 구성

프로젝트 최신순 · 수치는 git 저장소 기준(2026-10-15 집계, 외부 에셋 코드 제외)

One More Log

2026.10 ~ 프로토타입 개발 중 · 1인 개발

소개 실제 시간으로 타들어 가는 장작불을 꺼지지 않게 돌보는 불멍 시뮬레이션. Unreal Engine 5.8, C++17로 개발.

- 엔진과 분리한 C++17 결정론 연소 시뮬레이션 코어, 헤드리스 테스트 89건, 100시간 분량 따라잡기 계산 182ms
- Nanite · Lumen 숲 장면, 불 닿은 쪽부터 숯과 재로 변하는 장작 머티리얼, 불 크기에 비례하는 Niagara 불티

커밋 45회 · 핵심 방향은 직접 정하고, 구현은 AI 도구 위주

Digestor (이전 이름 Pill Agents)

2026.07 ~ 스토브 출시(v0.6.21) · 3인 팀: 개발·세부 기획·연출 담당 (팀원: 큰 틀 기획·모델링 1명, 모델링 1명)

소개 2~4인 온라인 협동 3D 액션 게임. 위 속에 들어간 알약 요원이 세균과 기생충 보스를 물리치고 탈출한다. Unity 6(URP), C#, Netcode for GameObjects, Unity Relay로 개발.

- 세부 기획과 설계를 직접 하고, 구현은 AI 도구 위주로 진행(Claude Code 설계 검토, Codex 구현, 별도 검증 단계)
- 서버 권위 구조, 기생충 보스 3페이즈와 팀이 공격 기회를 만드는 그로기 시스템 설계
- STOVE PC SDK 연동: 도전과제 39개, 클라우드 세이빙에 맞춘 진행 저장 방식

자동 테스트 1,287건 · 커밋 1,608회(112일)

Rat-errible Climb (쥐독한 등반)

2026.03 ~ 2026.06 · 스토브 인디 출시 · 3인 팀, 팀장 · 프로그래밍

소개 하수구 속 쥐가 함정을 피해 꼭대기까지 오르는 2D 등반 플랫포머. Unity 6(URP), C#, Cinemachine으로 개발.

- 모든 함정이 `TrapBase`를 상속해 경고, 작동, 대기 상태를 도는 함정 시스템 설계, 함정과 장애물 30종 구현
- Cinemachine 방 단위 카메라 전환, 사다리 · 물 · 끈적한 벽 상호작용
- 팀장으로서 맵 담당 팀원의 결과물을 직접 플레이해 보고 수정 방향을 피드백

C# 8,405줄, 이 중 약 60% 직접 작성(나머지는 AI 도구 보조)

SoulSync

2026.03 ~ 2026.07 · 1인 개발

소개 2인 온라인 협동 로그라이크. 불꽃·얼음 속성과 검·활 무기를 조합한 직업으로 몬스터 웨이브와 보스를 돌파한다. Unity 6, C#, Netcode for GameObjects, Unity Relay로 개발.

- 클라이언트는 요청만 보내고(ServerRpc) 서버가 결과를 기록하며(NetworkVariable) 연출만 전파(ClientRpc)하는 구조로 통일
- 보스 3종 패턴, 검 · 활 계열 스킬, 동료 부활, 솔로 모드

C# 23,150줄, 이 중 약 80% 직접 작성(나머지는 AI 도구 보조)

기술 · 학력 · 자격

언어 · 엔진	C#, C++ · Unity 6(URP, Cinemachine), Unreal Engine 5(독학)
멀티플레이	Netcode for GameObjects, Unity Relay, 서버 권위 구조
개발 도구	Git, Claude Code, Codex
학력	유한대학교 컴퓨터공학과 게임콘텐츠전공 (2022.03 입학, 2027.02 졸업 예정)
자격	정보처리기능사
병역	군필 (2023.01 ~ 2024.07)

자기소개서

1. 지원 동기와 입사 후 목표

플레이어가 직접 부딪히는 시스템을 만드는 프로그래머

게임플레이 프로그래머로 지원했습니다. 지금까지 만든 게임에서 맡은 일도 함정, 보스, 서버 판정처럼 플레이어가 직접 부딪히는 시스템이었고, 이런 시스템을 만들어 출시까지 가져가는 일을 계속하고 싶습니다.

Unity로 게임 세 편을 만들며 상태머신 기반 시스템과 서버 권위 멀티플레이를 다뤘고, 그중 두 편은 스토브에 출시했습니다. 지금은 언리얼 엔진 5를 독학하며 C++로 장작불 시뮬레이션 「One More Log」를 만들고 있어, Unity와 언리얼 어느 쪽 환경에서도 일할 수 있습니다.

입사 후 첫 1년은 팀의 코드 구조와 작업 규칙을 익혀 맡은 기능을 출시 빌드까지 책임지겠습니다. 그다음에는 셰이더를 직접 다룰 수 있게 공부해, 게임플레이와 화면 표현을 함께 맡는 프로그래머로 성장하겠습니다.

2. 직무 역량

3인 팀 게임을 스토브 인디에 출시한 경험

3인 팀의 팀장이자 프로그래머로 「Rat-errible Climb(쥐독한 등반)」을 만들어 스토브 인디에 출시했습니다. 하수구 속 쥐가 함정을 피해 꼭대기까지 오르는 2D 등반 플랫포머이고, Unity 6와 C#으로 개발했습니다.

가장 공들인 부분은 함정 시스템입니다. 함정이 늘 때마다 타이밍 코드를 따로 짜면 금방 관리가 안 될 것이 보였습니다. 모든 함정이 `TrapBase`를 상속받아 경고, 작동, 대기 세 상태를 돌게 만들고, 함정마다 다른 것은 타이밍과 넉백 값만 남겼습니다. 프로펠러, 가시, 고양이 발, 무너지는 발판 같은 함정과 장애물 30종을 이 틀 위에서 만들었고, 함정만 따로 돌려 보는 테스트 씬에서 타이밍을 먼저 맞춘 뒤 맵에 배치했습니다.

3. C++ · 언리얼 엔진 학습

언리얼 엔진 5와 C++로 만들고 있는 장작불 시뮬레이션

「One More Log」는 언리얼 엔진 5.8과 C++로 혼자 만들고 있는 장작불 시뮬레이션입니다. 실제 시간으로 타들어 가는 장작불을 꺼지지 않게 돌보는 게임으로, 자리를 비운 만큼 불이 정직하게 꺼진다는 핵심 규칙과 게임 방향은 제가 정했습니다. 구현은 AI 도구에 많이 맡기고 있습니다.

구조의 핵심은 연소 계산을 엔진에서 떼어 낸 것입니다. 연소 시뮬레이션은 언리얼에 의존하지 않는 C++17 코어로 만들고, 장작 위치를 정수 밀리미터와 0.1도 단위로 저장해 같은 입력이면 항상 같은 결과가 나오게 했습니다. 엔진 없이 돌아가는 테스트 89건으로 검증하고, 게임을 꺼 둔 100시간 동안의 연소도 182ms 만에 따라잡아 계산합니다.

밸런스도 이 코어로 측정합니다. 정해진 방식으로 장작을 넣는 봇을 돌려 불이 며칠을 버티는지 재는데, 지금은 7일 생존율이 목표 95%에 못 미치는 91.5%입니다. 작고 마른 장작이 먼저 바닥나는 것이 원인으로 나와, 다음 작업으로 연료 구성을 고칠 계획입니다. 화면은 Nanite와 Lumen으로 숲을 구성했고, 장작은 불이 닿은 쪽부터 그을리고 숯이 되었다가 재로 변하도록 머티리얼을 만들었습니다.

4. 멀티플레이 구현 경험

혼자 만든 2인 온라인 협동 게임

「SoulSync」는 기획부터 개발까지 혼자 만든 2인 온라인 협동 로그라이크입니다. 두 플레이어가 Unity Relay로 접속해 불꽃과 얼음 속성, 검과 활 무기를 조합한 직업으로 몬스터 웨이브와 보스를 상대하며, Unity 6, C#, Netcode for GameObjects를 썼습니다.

판정은 서버만 하도록 구조를 잡았습니다. 클라이언트는 공격이나 스킬을 요청만 보내고, 결과는 서버가 정해 NetworkVariable에 기록하며, 이펙트 같은 연출만 ClientRpc로 각자의 화면에 뿌립니다. 보스 3종과 검, 활 스킬을 모두 이 순서에 맞춰 만들었습니다. 두 사람이 멀리 떨어질수록 받는 피해가 커지는 규칙과 동료 부활을 넣어 협동하게 만들었고, 혼자 하는 사람을 위한 솔로 모드도 따로 만들었습니다.

5. AI 도구 활용과 문제 해결

구현은 AI 도구에 맡기고, 원인 확인과 재발 방지는 직접 챙기는 방식

AI 코딩 도구를 적극적으로 쓰지만, 무엇을 만들지와 결과가 맞는지는 제가 판단합니다. 「Rat-errible Climb」은 코드의 약 60%, 「SoulSync」는 약 80%를 직접 작성했습니다. 스토브에 출시한 「Digestor」는 구현을 AI 도구에 더 많이 맡긴 프로젝트로, 2~4인이 위 속에 들어간 알약 요원이 되어 세균과 기생충 보스를 물리치고 탈출하는 온라인 협동 3D 액션 게임입니다. Unity 6, C#, Netcode for GameObjects로 만들었습니다. 기획과 설계는 제가 정하고, Claude Code로 설계를 검토하고, Codex가 구현하며, 검증은 구현하지 않은 쪽에 맡깁니다.

애니메이션 임포터 코드를 고쳤는데 게임에 반영되지 않은 적이 있습니다. AI 도구와 함께 Unity 경고를 따라가 보니, 임포터 버전 번호를 올리지 않아 Unity가 예전 결과를 그대로 쓰고 있었습니다. 캐릭터 애니메이션 43개가 수정 전 상태였습니다. "임포터를 고치면 버전 번호도 올린다"는 규칙을 AI 도구가 작업할 때마다 읽는 프로젝트 지침 문서에 적었고, 이후 임포터를 두 번 더 고칠 때는 두 번 모두 버전 번호를 함께 올렸습니다.

6. 협업과 리더십

팀장으로서 팀원 결과물에 구체적으로 피드백한 경험

「Rat-errible Climb」에서 팀장을 맡았고, 맵 제작은 팀원이 담당했습니다. 팀원이 만든 맵을 직접 플레이해 보고, 클리어가 불가능한 구간, 지형에 가려 길이 보이지 않는 구간, 지나치게 어려운 5번 방처럼 문제가 되는 위치를 짚었습니다. 문제만 말하지 않고 발판을 추가하거나 배치를 바꾸는 식으로 고칠 방법까지 함께 전달했습니다. 팀원도 피드백을 잘 받아들여 이런 구간을 하나씩 고쳤습니다.

「Digestor」는 세 명이 함께 만듭니다. 저는 개발과 세부 기획, 연출을 맡고, 팀원 한 명은 큰 틀의 기획과 모델링을, 다른 한 명은 모델링을 맡고 있습니다. 모델이 늦어져도 개발이 멈추지 않게 코드로 만든 임시 모델을 먼저 넣고, 완성된 모델이 오면 슬롯에 끼워 바꾸는 구조를 만들었습니다. 받은 모델은 음식이 부서지는 단계와 액세서리가 붙는 위치를 캡처해 확인하고, 맞지 않는 부분은 고칠 방향을 정리해 전달합니다.

7. 강점과 보완점

강점은 끝까지 하는 끈기, 보완점은 여러 사람 앞에서 말하기

강점은 시작한 게임을 끝까지 만드는 끈기입니다. 「Rat-errible Climb」과 「Digestor」는 출시까지 마쳤고, 「SoulSync」는 혼자 만들면서도 멀티플레이와 보스전, 솔로 모드까지 완성했습니다. 보완점은 여러 사람 앞에서 말하는 것입니다. 팀원과 일대일로 이야기하는 데는 문제가 없지만 발표처럼 많은 사람 앞에 서면 긴장해서, 발표 연습을 꾸준히 하며 고쳐 가고 있습니다.